

Taskwarrior Nautical v6 - Systems Manual

1) Philosophy

Life is a highly complex endeavour so the focus should be on managing systems, not just tasks. Taskwarrior has a powerful system and Nautical gives you two complementary engines for describing recurring work:

Chains for period-based recurrence. Give it a duration like `3d` or `28h` - Nautical calculates the next due time from your completion.

Anchors for calendar-position recurrence. Specify patterns like `w:mon,wed` (Mondays and Wednesdays) or `m:2sat` (2nd Saturday) - Nautical walks your local calendar to find exact matches.

Both respect your time, preserve context across iterations, and stop exactly when you tell them to.

2) Reading this manual

Start with the installation and then jump to point 9 at the recipes library and write some test tasks to get a feel for the system.

When you want to find out more about the work under the hood and the capabilities then browse the other sections. Nautical is made to be reliable, intuitive and easy to use.

Best of luck and Godspeed.

Installation

Use the managed installer from the repository checkout. It installs the launcher, hooks, core runtime, and release metadata as one validated unit:

```
git clone --depth 1 https://github.com/catanadj/taskwarrior-nautical.git
cd taskwarrior-nautical
./nautical install --dry-run
./nautical install
```

The same command handles upgrades and safe re-runs. After updating the checkout, run `git pull --ff-only` followed by `./nautical install`. The installer uses `$TASKDATA` when set, otherwise `~/.task`; on Termux, pass `--launcher-path "$PREFIX/bin/nautical"` when that is the executable directory.

Register the shipped UDA definitions once:

```
cp uda.conf ~/.task/uda-nautical.conf
grep -Fqx "include $HOME/.task/uda-nautical.conf" "$HOME/.taskrc" 2>/dev/null || \
    printf '\ninclude %s\n' "$HOME/.task/uda-nautical.conf" >> "$HOME/.taskrc"
nautical doctor
```

Install optional Navigator and Rich dependencies with `python3 -m pip install -r requirements.txt`.

Configuration

Nautical reads an optional TOML file named `config-nautical.toml` or `nautical.toml`. It searches the Taskwarrior configuration and data directories, the `nautical_core/` directory, and the standard Nautical configuration directory. Set `NAUTICAL_CONFIG` to use an explicit file.

If the file is missing, sensible built-in defaults are used. Changes take effect on the next hook run.

Config precedence is: `config-nautical.toml` overrides built-in defaults.

Example: `config-nautical.toml`

```
# Random-selection identity salt and the IANA timezone used for calendar dates.
wrand_salt = "nautical|wrand|v4"
tz = "Australia/Sydney"                # e.g. UTC, Europe/Bucharest
season_hemisphere = "south"            # north or south
season_mode = "fixed"                  # fixed or astronomical; transition dates use local tz

# Trusted directories for file-backed dates; empty disables path resolution.
anchor_file_dir = ""                  # directory for anchor_file basenames/globs
omit_file_dir = ""                   # directory for omit_file basenames/globs

# File-backed anchor cache: true/false; the shipped template enables it.
enable_anchor_cache = true            # precompute file-backed hints
enable_uda_aliases = false            # true enables description aliases such as a: and am:
anchor_cache_dir = ""                 # empty uses Nautical's managed cache location
anchor_cache_ttl = 0                  # seconds; 0 means no expiry

# Hook and panel behavior.
chain_color_per_chain = true           # color timelines by chain
show_timeline_gaps = true              # include gaps between links
show_analytics = false                 # show completion analytics panels
analytics_style = "clinical"          # coach or clinical
analytics_ontime_tol_secs = 14400     # on-time tolerance in seconds
check_chain_integrity = false          # run extra chain integrity checks
debug_wait_sched = false              # include wait/scheduled carry diagnostics
recurrence_update_udas = []            # date UDAs to carry, e.g. ["next_review"]
panel_mode = "rich"                   # rich, live, fast, plain, line, minimal, text, quiet,
compact                                compact
live_panel_duration_ms = 160           # 0..1000 ms total reveal budget
live_panel_footer = "NAUTICAL"         # text in live panels; empty disables it
exit_progress = true                   # show on-exit drain progress
fast_color = true                      # use ANSI color in fast panels
outbox_drain_max_items = 32            # lifecycle plans drained per on-exit pass
max_chain_walk = 500                  # maximum links inspected by summaries

# Safety limits.
max_anchor_iterations = 128            # scheduler search limit (32..1024)
max_link_number = 10000               # maximum generated link number
sanitize_uda = false                  # remove control characters from UDAs
sanitize_uda_max_len = 1024           # sanitized UDA length (64..4096)
max_json_bytes = 10485760             # maximum hook input size

# Astronomical @t= event times and moon-phase anchors; requires Astral when used.
[astronomy]
default_location = "home"             # name from [astronomy.locations]
[astronomy.locations.home]
```

```

latitude = -33.86           # degrees, -90..90
longitude = 151.21          # degrees, -180..180
elevation = 0               # metres above sea level
timezone = "Australia/Sydney" # explicit civil timezone

# Optional named anchor presets. Use with anchor:"@name".
# TOML tables stay active until the next table, so keep top-level settings above this
point.
[anchor_presets]
payday = "m:15,-1bd"        # preset expression
workout = "w:mon,wed,fri"

[omit_presets]
holidays = "y:12-24..12-31" # preset omission expression
april = "y:apr"

# Optional named business calendar. Select it on a task with bc:work.
[business_calendar.work]
anchor = "w:mon..fri"        # one expression or an array
omit = ["y:01-01", "y:12-25"] # closed dates
# anchor_file = ["extra-open-days.csv"] # extra open dates, supports globs
# omit_file = ["holidays.csv", "company-closures-*.csv"]

```

Keys

chainID (required)

- New nautical task (has `anchor` or `cp`) gets `chainID = short(uuid)`.
- Spawned tasks inherit parent's ChainID.
- Modify won't overwrite an existing ChainID or stamp linked tasks.
- Hooks fetch chain members with the `chainID:<short>` filter instead of walking links.
- Nautical rejects recurrence mutations when the chain identity is incomplete.
Every chain must have an explicit `chainID`; manually clearing or changing it is not a supported way to edit a recurrence.

enable_anchor_cache (bool, built-in default: false; shipped template: true)

When true, Nautical precomputes and writes **ACF (Anchor Canonical Form) + anchor hints** (e.g., first due, timeline preview) to a cache file for near-instant previews. When false, no cache file is written and ACF (Anchor Canonical Form) is only kept in memory for the session (never stored as a UDA).

anchor_cache_dir (string, default: empty)

Where to write the cache when `enable_anchor_cache=true`. When empty, Nautical tries a cache directory inside `nautical_core/`, then `TASKDATA`, then the platform cache directory. An explicit path can be relative or absolute and must be writable.

anchor_file_dir (string, default: empty)

Trusted directory for task-level `anchor_file:` basenames. Nautical resolves `anchor_file` only inside this directory and rejects paths like `../calendar.csv` or `/tmp/x.csv`.

omit_file_dir (string, default: empty)

Trusted directory for task-level `omit_file:` basenames. Nautical resolves `omit_file` only inside this directory and rejects paths like `../holidays.csv` or `/tmp/x.csv`.

business_calendar (TOML table, optional)

Each `[business_calendar.<name>]` section defines one calendar that tasks select with the string UDA `bc:<name>`. The calendar's open dates are:

```
(anchor union anchor_file) - (omit union omit_file)
```

The four fields accept either a string or an array of strings. `anchor_file` uses `anchor_file_dir`, while `omit_file` uses `omit_file_dir`; file expressions support the same `*` and `?` patterns as task-level file sources. Patterns must match at least one file.

```
[business_calendar.work]
anchor = "w:mon..fri"
omit = ["y:01-01", "y:12-25"]
# anchor_file = ["extra-open-days.csv"]
# omit_file = ["holidays.csv", "company-closures-*.csv"]
```

```
task add "Submit payroll" anchor:"m:-1bd@t=16:00" bc:work
```

If `bc` is empty, Nautical uses its built-in Monday-Friday calendar. Calendar names are case-insensitive and stored in normalized form; unknown names are rejected with the configured names. The selected calendar controls business-day ordinals, filters, rolls, offsets, file modifiers, previews, and spawned links.

Calendar definitions must have stable date membership. `/N`, random selectors, `@t=`, business-day ordinals, and business-day modifiers are rejected inside calendar fields because a calendar cannot depend on the business days it is defining. Nautical includes the resolved rules and file dates in recurrence cache keys, so calendar changes do not reuse stale hints.

NAUTICAL_DNF_DISK_CACHE (env, default: enabled)

On-add JSONL cache for parsed anchors. Set `NAUTICAL_DNF_DISK_CACHE=0` to disable.

max_anchor_iterations (int, default: 128)

Upper bound used by anchor search loops.

max_link_number (int, default: 10000)

Upper bound for auto-incremented chain link numbers.

sanitize_uda (bool, default: false)

When true, sanitize string fields by removing control characters and clamping length.

sanitize_uda_max_len (int, default: 1024)

Maximum length for sanitized string values.

max_json_bytes (int, default: 10485760)

Maximum size accepted from hook stdin.

NAUTICAL_CLEAR_CACHES (env, default: disabled)

When set to `1`, clear in-process LRU caches after anchor parsing.

DST policy

When Nautical converts a local wall-clock time to UTC (e.g., anchor `@t` times), it applies a deterministic DST policy:

- Ambiguous times (fall back) choose the earlier occurrence.
- Nonexistent times (spring forward) shift forward by the size of the clock change.

on-modify toggles (config)

Hook behavior can be tuned via `config-nautical.toml`:

- `chain_color_per_chain` (bool, default: false)
- `show_timeline_gaps` (bool, default: true)
- `show_analytics` (bool, default: true)
 - Set `show_analytics = false` to disable analytics output.
- `analytics_style` ("coach" or "clinical", default: clinical)
 - `analytics_style` has no "off" value; disable via `show_analytics = false`.
- `analytics_ontime_tol_secs` (int seconds, default: 14400)
- `debug_wait_sched` (bool, default: false)
- `recurrence_update_udas` (list or CSV string, default: empty)
 - Date-type UDA fields to carry to the next link using the same wall-clock delta rule as `wait`/`scheduled`.
 - Example: `recurrence_update_udas = ["rappel", "next_review"]`
 - Also accepted: `recurrence_update_udas = "rappel,next_review"`
- `check_chain_integrity` (bool, default: false)
- `panel_mode` ("rich", "live", "fast", "line", "compact", "minimal", or "text"; default: "rich")
 - "live" reveals panel rows over at most 160 ms on interactive Rich terminals, then leaves the complete panel visible.
 - Captured output and terminals without cursor control automatically use the static panel.
 - Panels using more than 75% of the terminal height, or leaving fewer than three spare lines, settle immediately to avoid scrolling and flicker.
 - Multiline values reveal one line at a time under a stable label, with timing eased toward the final settled frame.
- `live_panel_duration_ms` (integer milliseconds, default: 160; range: 0..1000)
 - Sets the total reveal budget, not a delay per row. Use `0` to retain live styling without motion.
 - Error panels render immediately, warning panels use at most half the budget, and only the first eligible panel in a hook execution animates.
- `exit_progress` (bool, default: true)
- `fast_color` (bool, default: true)
- `max_chain_walk` (int, default: 500)

3) Notation and Conventions

This section is the “grammar” Nautical understands.

3.1 Engines & where they read time from

- **Chains** (`cp`) - Use one **period** (e.g., `3d`, `28h`, `P2W`) or a sequence of periods (e.g., `3d, 20d, 7d`). Next link is computed from the **completion time** (and may preserve wall-clock; see §3.2).

- **anchors** (`anchor`, `anchor_file`) - Use **calendar positions** and/or **file-backed dates**. Nautical builds one local recurrence stream from:
 - `anchor` expressions
 - `anchor_file` explicit dates
 and then applies omission rules from `omit` / `omit_file`.

3.2 Dates, times, and time zones

- **Local vs UTC:** All calendar reasoning (anchors) is done on **local dates**; panels display local time. Internally, times persist in UTC.
- **Seed time:** The **first link's** `due:` **time** becomes the wall-clock reference.
- If your period is a **multiple of 24h** (e.g., `2d`, `1w`), Nautical keeps that wall-clock (due time).
- To force exact addition for an interval near a multiple of 24 hours, add or subtract one second, for example `cp:24h+1s`. The resulting interval is exactly 24 hours and one second from completion.
- For other periods (e.g., `28h`, `33h`) Nautical does an **exact add from end**.
- **Per-term time:** You can attach time to specific anchor terms with `@t=HH:MM` (e.g., `w:mon@t=09:00,fri@t=15:00`). With an explicit astronomy profile, `@t=sunrise`, `@t=sunset`, `@t=dawn`, `@t=dusk`, `@t=moonrise`, and `@t=moonset` resolve the event for each matching date. You can also place `@t=` after a parenthesized expression to give every branch the same time: `(w:mon | m:last-fri)@t=09:00`. If omitted, Nautical uses the seed link's `due:` time.
- **Fixed yearly date style:** `y:MM-DD` only (same MM-DD style as Taskwarrior dates). Ordinal selectors `y:dN` and `y:wN` are independent of date format.

3.3 Anchor tokens you can use (by family)

Weekly

- **Weekdays:** `mon`, `tue`, `wed`, `thu`, `fri`, `sat`, `sun`.
- **Lists:** `w:mon,fri`.
- **Ranges:** `w:mon..wed` (inclusive).
- **Stepped cadence:** `w/2:mon,tue` (every 2 weeks on Mon & Tue), `w/3:fri` (every 3rd week on Fri).
- **Random:** `w:rand` (one random day each week).
- **Counted random:** `w:2rand` (two distinct random days each week).

Monthly – by date

- **Specific days:** `m:1`, `m:15`, `m:31`.
- **Last day:** `m:-1`.
- **Lists:** `m:1,15,-1`.
- **Ranges/buckets:** `m:1..7` (days 1–7), `m:22..28`, etc.
- **Business-day ordinals:** `m:5bd` (5th business day), `m:15bd`.
- **Stepped cadence:** `m/2:-1` (every 2 months on last day), `m/3:1` (every 3 months on the 1st).
- **Random:** `m:rand` (one random day each month).
- **Counted random:** `m:3rand` (three distinct random days each month).

Monthly – by weekday position

- **Nth weekday:** `m:1mon` ... `m:5sun` (1st–5th).

- **Last weekday:** `m:last-fri`, `m:last-mon`, etc.
- **Lists:** `m:2mon,4thu`. (2nd Monday, 4th Thursday)
- **Stepped cadence:** `m/2:2sat` (every 2 months, 2nd Saturday), `m/4:last-fri`.

Yearly

- **Specific dates:** `y:05-20`.
- **Calendar-day ordinals:** `y:d100` (100th day), `y:d-1` (last day). Positive values run from `d1` through `d366`; negative values run from `d-1` through `d-366`.
- **ISO-week ordinals:** `y:w20` (all seven days of ISO week 20), `y:w20 + w:mon` (its Monday), `y:w-1 + w:fri` (Friday of the final ISO week).
- **Ordinal lists and ranges:** repeat the prefix for every item or range endpoint, for example `y:d1,d100,d-1` and `y:w10..w13`.
- **Lists:** `y:01-15,04-15,07-15,10-15`.
- **Ranges (inclusive):** `y:01-20..01-27` (Jan 20–27), `y:04-20..05-15` (Apr 20–May 15).
- **Random month pick:** `y:10-rand` (one random day in October, deterministic per chain).
- **Random year pick:** `y:rand` (one random day each year).
- **Counted random:** `y:2rand` (two distinct random days each year).
- **Leap day:** `y:02-29` (appears only in leap years).
- **Stepped cadence:** `y/3:06-07` (every 3 years on 7 of June)

3.4 Modifiers (@...) and what they do

Attach modifiers to **individual terms** (right after them). Multiple modifiers can be chained. Nautical applies date transforms in a fixed order regardless of their textual order: **roll**, **calendar-day offset**, then **business-day offset**.

- **Time of day:** `@t=HH:MM` - sets the time for that term only.
 - Example: `w:mon@t=09:00,fri@t=15:00` (Monday at 9, Friday at 15)
 - A time can also follow a parenthesized expression: `(w:mon | m:last-fri)@t=09:00`. Nautical applies it to every branch in the group.
 - Parenthesized OR groups can share `@t=`, rolls, filters, calendar-day offsets, and business-day offsets. Example: `(y:12-24 | y:12-31)@pbd@-1bd@t=09:00`.
 - Shared `@t=` also works on groups containing `+`. Date modifiers on `AND` groups must remain attached to individual atoms because distributing a roll across an intersection can create false matches.
 - Do not combine a group-level time with an existing inner `@t=`. Use either one shared group time or explicit per-term times.
- **Random time windows:** `@t=rand(HH..HH)` selects one minute uniformly from the inclusive window. Add `/N` to select one minute from each of `N` evenly sized buckets: `@t=rand(06..18/3)` produces three deterministic times between 06:00 and 18:00. Minutes may be included explicitly, for example `rand(06:30..18:15/2)`.
- Random windows may cross midnight, such as `@t=rand(22:30..02:30/3)`; the later slots belong to the following local date. Selection is reproducible for a chain and anchor date, so previews, completion, Navigator, and reconciliation agree without storing random state.
- Random time windows use minute precision and require a Nautical chain identity. The optional `/N` count must fit inside the window and the normal time-slot resource limit. Use a regular time window or a time list when exact, repeatable clock times are required.
- **Business-day rolls** (for date-based terms):
 - `@nbd` - keep an open date unchanged; otherwise roll to the **next business day**.
 - `@pbd` - keep an open date unchanged; otherwise roll to the **previous business day**.
 - `@nw` - keep an open date unchanged; otherwise roll to the **nearest business day**.

- `@bd` - keep **business days only** for random or bucket picks.
- Without `bc`, business days are Monday through Friday. With `bc:<name>`, these modifiers use that named calendar.
- **Day offsets:**
 - `@+Nd` - shift the matched date **forward by N days** after any roll/filter is applied.
 - `@-Nd` - shift the matched date **backward by N days** after any roll/filter is applied.
 - Examples: `y:04-25@+2d` (Apr 25 each year, 2 days later), `y:04-25@-2d` (Apr 25 each year, 2 days earlier), `m:1@nbd@+1d` (1st rolled to next business day, then moved one day later).
- **Business-day offsets:**
 - `@+Nbd` - move **forward by N business days**, excluding the matched date itself.
 - `@-Nbd` - move **backward by N business days**, excluding the matched date itself.
 - Without `bc`, business days mean Monday through Friday. With `bc:<name>`, the task uses that configured calendar, including user-defined closures and exceptional open dates.
 - Under the default calendar, `y:04-24@+1bd` moves a Friday match to Monday, and `m:-1@pbd@-2bd` rolls a closed month-end to the previous weekday before moving back two additional business days.
- **Specific weekday rolls** (apply to a date to find the next/previous named weekday):
 - `@next-mon`, `@prev-fri`, `@next-sat`, `@prev-mon`, etc.
 - For example anchor:"y:12-31@prev-fri" will match the Friday before the end of the year (even if 12-31 is a Friday).

Scope notes:

- Rolls are most meaningful for **monthly and yearly day** terms (e.g., `m:1@nbd`, `y:02-28@nw`, `y:03-05@-10d`).
- Without a named calendar, weekly terms such as `w:mon` already select an open weekday, so applying `@nbd` is redundant. A named calendar may close that date.
- Calendar-day and business-day offsets apply after rolls, so they compose naturally with `@t=...`, `@nbd`, `@pbd`, `@nw`, `@next-mon`, and `@prev-fri`.
- `m:5bd` is a monthly selector meaning “the 5th business day of the month.” By contrast, `@+5bd` is a modifier that moves an already matched date forward five business days.

3.5 Logic & grouping

- **AND:** `+` - both sides must match.

Example: `w:mon + m:1,15` → Mondays that are also the 1st or 15th.

- **OR:** `|` - either side may match.

Example: `m:1sat | m:3fri`.

- **Parentheses:** group sub-expressions.

Example: `(m:1..7 + m:rand@bd) | (m:8..14 + m:rand@bd)`.

Parentheses can also share one time across the complete expression:

Example: `(w:mon | m:last-fri)@t=09:00`.

- **Precedence:** `+` (AND) binds **tighter** than `|` (OR). Use parentheses to make intent explicit.
- **Whitespace:** optional; ignored inside patterns. When your shell could parse `+` or `|`, **quote** the pattern:

Example: `task add "Workout" anchor:"w:mon,wed,fri + y:apr"`.

Practical grammar reference

This is the short version of how to read Nautical expressions.

For `cp`, the grammar is period based:

```
cp = period
    | period,period,...
    | rand(period..period)
    | period~spread
```

Examples:

- `cp:3d` - one fixed period.
- `cp:"3d,20d,7d"` - repeat these periods in order.
- `cp:"rand(3d..7d)"` - pick one chain-scoped deterministic random period in that range for each link.
- `cp:"14d~2d"` - same idea as `rand(12d..16d)`, easier to read as base plus/minus spread.

For `anchor` and `omit`, the grammar is calendar based:

```
expression = branch ("|" branch)*
branch     = factor ("+" factor)*
factor     = atom | "(" expression ")" ["@in-" scope "=" positions] ("@" modifier)*
atom       = family ["/N"] ":" spec ["@" modifier]*
family     = "w" | "m" | "y"
scope      = "week" | "month" | "quarter" | "year" | "spring" | "summer" | "autumn" |
"winter"
positions  = position ("," position)*
```

Inside `y:`, `dN` selects a calendar-day ordinal and `wN` selects an ISO-week ordinal. Negative ordinals count backward from the end of the calendar year or ISO year.

Read this in plain English:

- `|` means **either/or**.
- `+` means **must also match**.
- `+` binds tighter than `|`.
- Parentheses make the grouping explicit.
- A comma belongs **inside one atom**. It is a list, not a top-level OR operator.

Examples:

- `w:mon,wed,fri` means Mondays, Wednesdays, or Fridays.
- `w:mon,wed,fri + y:apr` means Mondays, Wednesdays, or Fridays **in April**.
- `w:mon | w:wed | w:fri + y:apr` means Mondays, or Wednesdays, or Fridays in April. This is different because `+` binds before `|`.
- `(w:mon | w:wed | w:fri) + y:apr` is the explicit form of the grouped April version.
- `m:rand + w:sat,sun` means one random weekend date per month.
- `m:rand + (w:sat | w:sun)` means one random Saturday branch and one random Sunday branch, so it can produce two dates per month.

Modifiers are attached with `@`:

- `w:mon@t=09:00` - Mondays at 09:00.
- `m:1@nbd` - the 1st, rolled to the next business day if needed.
- `m:-1@pbd@-2bd` - month-end rolled backward if necessary, then shifted two more business days earlier.
- `y:12-31@prev-fri` - the Friday before Dec 31.
- `(w:mon | w:fri)@t=09:00` - both branches share the same time.
- `moon:full` - next local calendar date nearest the full moon.
- `w:fri@moon=full` - Fridays filtered to full-moon dates.
- `(moon:full + w:fri)@t=20:00` - full moon intersected with Friday at 20:00.

Moon phases use `new`, `first-quarter`, `full`, and `last-quarter`; common aliases such as `full-moon` are accepted. Configure an astronomy location with an explicit timezone. If Astral or the profile is missing, hooks show an actionable error and reconcile fails closed rather than guessing a date.

Parenthesized OR groups accept the same modifiers as atoms. Shared `@t=` also works on groups containing `+`, but date modifiers on `AND` groups are rejected; attach those modifiers to individual atoms instead. Grouped date modifiers also cannot be layered over date modifiers already present inside a branch.

`omit` uses the same date grammar as `anchor`, but it is date-based only. Time modifiers such as `omit:"w:mon@t=09:00"` are rejected because omissions remove dates, not individual timed instances.

3.5.1 Positional selection

Use positional selection when a recurrence depends on a date's position within the complete set of matches for a week, month, quarter, or year.

```
(candidate expression)@in-<scope>=<positions>
```

Evaluation order is fixed:

```
collect matches in the period -> select positions -> apply modifiers
```

The candidate expression must be parenthesized. Positions can be `first`, `last`, a positive ordinal such as `3rd`, a reverse ordinal such as `2nd-last`, or a comma-separated list.

Selector	Period	Maximum position
<code>@in-week</code>	Monday through Sunday	7
<code>@in-month</code>	Calendar month	31
<code>@in-quarter</code>	Calendar quarter	92
<code>@in-year</code>	Calendar year	366
<code>@in-season</code>	Each fixed seasonal window	92

Examples:

```
task add "Weekly closeout" anchor:"(w:mon | w:wed | w:fri)@in-week=last"
task add "Twice-monthly review" anchor:"(w:tue | w:thu)@in-month=first,last"
task add "Quarter handoff" anchor:"(w:mon)@in-quarter=last@+1bd"
task add "Annual checkpoint" anchor:"(w:mon)@in-year=10th@t=09:00"
```

Selection is performed before modifiers. In the quarter example, Nautical first finds the last Monday in the calendar quarter and then moves that selected date forward by one business day. The transformed result may fall in the next period. Business-day rolls and offsets use the task's `bc` calendar.

If a requested position does not exist, it contributes no occurrence. If the entire period has no selected date, Nautical advances to the next period. For example, `(y:02-29)@in-year=first` skips non-leap years.

Supported post-selection modifiers are `@t=`, `@+Nd`, `@-Nd`, `@+Nbd`, `@-Nbd`, `@pbd`, `@nbd`, `@nw`, and named weekday rolls such as `@next-mon` or `@prev-fri`.

The candidate set must remain deterministic. Random candidate atoms, candidate-side modifiers, nested positional selectors, and `@bd` after a selector are rejected. Use ordinary deterministic atoms such as `w:mon..fri` when defining a candidate set.

3.5.2 Seasonal selection

Seasonal scopes use the same positional syntax with inclusive calendar windows. The default `season_mode = "fixed"` uses the deterministic windows below. Set `season_mode = "astronomical"` to derive each boundary from the year's equinoxes and solstices. Astronomical transition instants are converted to the configured `tz`; the transition's local calendar date starts the new season, so every date belongs to exactly one window.

`@in-season` is the broad form when the same position should be selected in each season. For example, `(w:mon)@in-season=1st` means the first Monday of spring, summer, autumn, and winter. The active `season_hemisphere` setting controls the hemisphere mapping. `season_mode = "fixed"` (the default) uses the deterministic windows below; `season_mode = "astronomical"` derives the boundaries from equinoxes and solstices.

Selector	Fixed window (default)
<code>@in-spring</code>	March 1 through May 31
<code>@in-summer</code>	June 1 through August 31
<code>@in-autumn</code>	September 1 through November 30
<code>@in-winter</code>	December 1 through February 28/29

The candidate expression must be parenthesized. Nautical collects all matches inside that seasonal window, selects the requested position, and only then applies time, roll, or offset modifiers. Winter is identified by its December start year, so a winter beginning in December 2026 ends in February 2027.

```
# First and last Monday of each spring.
task add "Spring planning" anchor:"(w:mon)@in-spring=first,last"

# Last Friday of each winter, at 09:00.
task add "Winter review" anchor:"(w:fri)@in-winter=last@t=09:00"

# First weekday of summer, then move one business day forward.
```

```
task add "Summer handoff" anchor:"(w:mon..fri)@in-summer=first@+1bd"

# The first leap day that occurs inside a winter window.
task add "Leap-day check" anchor:"(y:02-29)@in-winter=first"
```

In fixed mode these are deterministic calendar definitions. In astronomical mode the transition date in the configured timezone starts the new season, so the date windows remain contiguous and non-overlapping. A requested position that does not exist contributes no occurrence; for example, the leap-day example advances to the next winter containing February 29.

3.6 Modes (anchor backfill semantics)

Attach via UDA: `anchor_mode:skip` (default), `all`, or `flex`.

- **skip** - do **not** backfill missed anchors; jump to the next future match. (training or vitamins)
- **all** - **backfill every** missed anchor before moving forward (useful for bills).
- **flex** - skip backlog **once**, then behave like **all** going forward.
- Explanation: This is used when the mode of a task is 'all' and you don't want to backfill the overdues but want to continue doing the task going forward; in that instance you modify the anchor_mode to 'flex' and on the next completion of the task, Nautical is going to jump overdues and go to the next anchor match and change the mode to 'all'. This is a one-off convenience mode.

3.7 Unified anchor recurrence (`anchor`, `anchor_file`, `omit`, `omit_file`)

Anchor recurrence now has two inclusion sources and two exclusion sources:

- inclusion:
 - `anchor`
 - `anchor_file`
- exclusion:
 - `omit`
 - `omit_file`

Nautical evaluates them as:

- `(anchor U anchor_file) - (omit U omit_file)`

This means:

- `anchor` and `anchor_file` can be used **together** on the same task.
- `omit` and `omit_file` are applied **after** both inclusion sources are merged.
- omitted dates stay visible in timeline-style feedback as explicit skipped rows.

Inclusion sources

- `anchor` uses the full anchor grammar described above.
- `anchor_file` loads explicit dates from one or more files in `anchor_file_dir`.
- If both are present, Nautical takes the **union** of both occurrence streams.
- If both sources produce the same local datetime, Nautical deduplicates it.

Anchor presets

Reusable anchor expressions can be defined in `config-nautical.toml`:

```
[anchor_presets]
payday = "m:15,-1bd"
workout = "w:mon,wed,fri"

[omit_presets]
holidays = "y:12-24..12-31"
april = "y:apr"
```

Use them with `@name`:

```
task add "Pay bills" anchor:"@payday"
task add "April workouts" anchor:"@workout + y:apr"
task add "Workout except April" anchor:"@workout" omit:"@april"
```

Simple preset panels show the expansion:

Preset @payday → m:15,-1bd

`[anchor_presets]` and `[omit_presets]` are intentionally separate namespaces. The same `@name` can be defined differently for inclusion and omission.

Accepted `anchor_file` formats:

- Plain text lines:
 - `YYYY-MM-DD`
 - `YYYY-MM-DD..YYYY-MM-DD`
 - blank lines and `#` comments are ignored
- CSV with a `date` column:
 - column order does not matter
 - extra columns are ignored
 - repeated dates are deduplicated

`anchor_file` is basename-only. Nautical resolves it only inside `anchor_file_dir`. Join multiple sources with `|`, group them with parentheses, or use `*` and `?` file-name patterns. Exact `*` and `*.*` both include every non-hidden regular file in the configured directory; paths, recursive `**`, and character classes are rejected.

```
task add "Regional events" anchor_file:"north.csv@t=09:00 | south.csv@t=15:00"
task add "Team events" anchor_file:"team-*.csv@t=09:00"
task add "Prepared events" anchor_file:"(public.csv | company.txt)@-1d@t=12:00"
```


File-backed dates accept the same date transforms, including `@+Nbd` and `@-Nbd`. For example, `anchor_file:'events.csv@-2bd@t=12:00'` moves each file date two business days earlier before creating its timed occurrence. `omit_file` also accepts business-day offsets, but remains date-based and rejects `@t=`.

Exclusion sources

- `omit` and `omit_file` apply to **anchor recurrence only**.
- `omit` uses the **same anchor grammar** as `anchor`.
- `omit_file` loads explicit blocked dates from a file in `omit_file_dir`.
- Omitted dates are **skipped, not rolled**. Nautical keeps searching until it finds the next valid anchor date.
- `omit` is **date-based only**. Time modifiers like `@t=09:00` are rejected.
- If `omit` or `omit_file` removes every future match, Nautical fails cleanly instead of looping.

Accepted `omit_file` formats:

- Plain text lines:
 - `YYYY-MM-DD`
 - `YYYY-MM-DD..YYYY-MM-DD`
 - blank lines and `#` comments are ignored
- CSV with a `date` column and an optional `description` column:
 - column order does not matter
 - extra columns are ignored
 - repeated dates are deduplicated

Examples:

- Skip all Wednesdays from a weekly pattern:
 - `anchor:w:mon,wed,fri omit:w:wed`
- Merge expression and file-backed dates:
 - `anchor:w:tue,fri anchor_file:events.csv@-1d@t=12:00,18:00`
- Skip one yearly date window from a monthly random Saturday pattern:
 - `anchor:'m:rand + w:sat' omit:'y:12-24..12-31'`
- Skip a recurring monthly anchor on a specific yearly date:
 - `anchor:m:1 omit:y:01-01`
- Skip dates from a file in `omit_file_dir`:
 - `anchor:'m:rand + w:sat' omit_file:holidays.csv`
- Use all four sources together:
 - `anchor:'w:tue,fri | y:05-05' anchor_file:'events.csv@-1d@t=12:00,18:00' omit:'y:04-28..05-05' omit_file:holidays.csv`

3.8 Caps (hard stops)

- By count:** `chainMax:N` - stop at the **N-th** link; panels mark second-to-last and last.
- By date:** `chainUntil:YYYY-MM-DD[Thh:mm]` - include anchors $\leq$ **this local moment**; last link is the final anchor on/before that time.
- Both set:** Nautical stops at **whichever comes first**.

- **Overdue rule:** If a link is overdue, **nothing spawns** until you complete it. An intentional deletion before its native `until` ends the chain; automatic expiration at `until` advances to the next occurrence.

3.8.1 Per-occurrence expiration

Taskwarrior's native `until` expires the current task; it does not limit the Nautical chain. By default, Nautical carries the expiration by calendar: each child gets the same calendar-day distance from its own `due` (or scheduled-only `scheduled`) and the same local clock time. `chainMax` and `chainUntil` remain the hard limits for the whole chain.

For Nautical tasks, native `until` must be strictly later than the effective `due` or scheduled-only `scheduled`. Nautical rejects invalid windows when the task is created, promoted into recurrence, modified, or completed.

Native `until` cannot be combined with anchor modes `all` or `flex`, because those modes retain every missed occurrence. Use `anchor_mode:skip` for per-occurrence expiration, or `chainUntil` when the whole chain should end.

```
task add "Take the trash out" due:today until:eow cp:7d chainUntil:eoy
```

If the task remains unfinished at the end of the week, Taskwarrior deletes that occurrence and Nautical advances from its original `due` (or scheduled-only `scheduled`) to the next one. The **Expiration** row explains the calendar policy (for example, `Same day at 23:00` or `1 calendar day later at 09:00`), and **Next expires** shows the concrete timestamp.

Use `+1s` to opt into an exact elapsed expiration window when retaining the duration matters more than retaining the calendar placement:

```
# Every generated occurrence expires exactly as long after its due as this one.
task add "Short response window" due:today+09:00 until:eod+1s cp:1d
```

The panel labels this as `Exact · ... after occurrence`. Without `+1s`, use the calendar default; it is usually the natural choice for daily and weekly work.

When hooks are unavailable or expiration happens on a synced client, use `nautical reconcile` to review the recovery and `nautical reconcile --apply` to apply it.

3.8.2 Manual stop of the chain

- Modification of the "chain" UDA to **off**. When this is executed and the task is completed, Nautical isn't going to spawn the next instance.
- **Intentional deletion before native** `until`. Automatic expiration at `until` advances the chain instead.

3.8 Random behavior (deterministic)

- `w:rand` - one random day per ISO week.
- `m:rand` - one random day each month.
- `m:rand@bd` - one random **business day** each month.

- `m:rand + w:mon,sat` - one random date each month chosen from all Mondays and Saturdays. The comma-separated weekday list is one candidate pool, so only one task is produced per month.
- `m:rand + (w:mon | w:sat)` - two independent random branches: one Monday and one Saturday each month. Use explicit `|` only when you want both.
- `y:MM-rand` - one random day in that month each year.
- `y:rand` - one random day in the year.

Counted random uses the same syntax in every family:

- `w:Nrand` - select `N` distinct random dates per eligible ISO week.
- `m:Nrand` - select `N` distinct random dates per eligible month.
- `y:Nrand` - select `N` distinct random dates per eligible year.

Examples:

- `w:2rand` - two random days each week.
- `m:3rand + w:mon..fri` - three random weekdays each month.
- `y:2rand + y:apr,jul,oct` - two random dates each year selected from April, July or October.

Random anchors behave like independent dice rolls, but the result is reproducible. Each draw is derived from the random algorithm version, `wrand_salt`, `chainID`, normalized expression, and recurrence period.

- Two tasks using the same random anchor can pick different dates in the same period because their `chainID` values differ.
- The same chain and period always produce the same date or counted selection. Preview, completion spawning, timeline reconstruction, omit evaluation, retries, and sync therefore agree without storing mutable random state.
- Changing `wrand_salt` deliberately produces a new sequence of future random dates. Keep it stable if existing chains must retain their current sequence.
- Constrained random anchors choose uniformly from all matching dates. For example, `y:rand + y:apr,jul,oct` rolls among every valid date in those three months.
- Counted selections are made without replacement and then ordered chronologically. A date cannot be selected twice within the same period.
- Constraints and omissions are applied before the draw. If a selected date is omitted, Nautical redraws from the remaining candidate pool so the requested count is retained.
- If an eligible period has fewer than `N` remaining candidates, that period is skipped.
- Counted random composes with stepped cadence and shared times, for example `m/2:2rand` and `(m:2rand + w:mon..fri)@t=09:00`.
- Repeats and streaks are valid random outcomes. Nautical does not force balancing or shuffle eligible dates into a visible cycle.
- check the anchor library for more examples.

3.9 Validation & edge cases

- **Strict parsing:** malformed tokens and impossible dates are rejected (e.g., `m:1,1as5`, `y:02-30`, and out-of-range ordinals). `y:02-29` is valid and simply contributes no occurrence in a non-leap year.
- **Count limits:** weekly random counts cannot exceed 7, monthly counts cannot exceed 31, and yearly counts cannot exceed 366. Business-day filters impose smaller practical limits.
- **Case-insensitive** tokens (`Mon`, `MON`, `mon` all work); prefer lowercase.

- **Business day definition:** Monday through Friday when `bc` is empty; otherwise the selected named business calendar defines open dates.
- **Quoting:** Always quote patterns with spaces, `+`, or `|`.
- **One recurrence engine per task:** Use either `cp` or **anchor recurrence** on the same task.
 - Anchor recurrence may use any combination of:
 - `anchor`
 - `anchor_file`
 - `omit`
 - `omit_file`

4) UDAs setup

```
# #####
# #####
# #####

## Classic Chain Recurrence

uda.cp.type=string
uda.cp.label=Chain Period
uda.chain.type=string
uda.chain.label=Chain Status
uda.chain.values=on,off
uda.chain.default=off

## Advanced Anchor Recurrence

uda.anchor.type=string
uda.anchor.label=Anchor
uda.anchor_file.type=string
uda.anchor_file.label=Anchor File
uda.anchor_mode.type=string
uda.anchor_mode.values=flex,all,skip
uda.anchor_mode.default=skip
uda.anchor_mode.label=Anchor Mode

uda.bc.type=string
uda.bc.label=Business Calendar

uda.omit.type=string
uda.omit.label=Omit
uda.omit_file.type=string
uda.omit_file.label=Omit File
```

```
## Limits
```

```
uda.chainMax.type=numeric
uda.chainMax.label=Chain Max
uda.chainUntil.type=date
uda.chainUntil.label=Chain Until
```

```
## Lineage
```

```
uda.prevLink.type=string
uda.prevLink.label=Previous Link
uda.nextLink.type=string
uda.nextLink.label=Next Link
uda.link.type=numeric
uda.link.label=Link Number
uda.chainID.type=string
uda.chainID.label=ChainID
```

```
#
```

5) Core behavior

- **One pending link.**
- **Sync-aware completion.** Stable child IDs and equivalent-child checks prevent future duplicate chain slots when current hooks run on every synced device.
- **Overdue pauses.** If a link is overdue, Nautical waits. Complete it to move on, or intentionally delete it before native `until` to stop the chain. A native `until` expiration advances to the next occurrence.
- **Copies your context.** Project, tags, UDAs, dependencies, and **annotations (timestamps kept)** travel to the next link.
- **Caps are hard stops.** `chainMax` (by count) / `chainUntil` (by date). Panels mark second-to-last and last link.
- **Predictable time policy.** Multiples of 24 hours preserve wall-clock time; other spans add exactly from completion.

6) Chains (Classic Chain Recurrence)

6.1 Period syntax you can use

The simplest chain uses one period in `cp`.

```
task add "Trim the grass" cp:12d due:tomorrow+9h
```

This means:


```
#1 is due tomorrow at 09:00
# When #1 is completed, Nautical creates #2 due 12 days later
# When #2 is completed, Nautical creates #3 due 12 days later
```

For a single period, every link uses the same interval. This is the classic Nautical chain: completion advances the chain by one configured period.

Common period forms:

- Short forms: `12d`, `2w`, `28h`, `90m`.
- ISO-8601 works too: `P12D`, `P2W`, `PT28H`.

Advanced period forms:

- Sequence forms: `3d,20d,7d,10d,3d`.
- Random range forms: `rand(3d..7d)`.

6.2 Sequential periods

Once the single-period model is clear, `cp` can also hold a comma-separated sequence of periods:

```
task add "Insect treatment" cp:"3d,20d,7d,10d,3d" due:today
```

Nautical uses one period per completed link:

```
#1 -> #2: 3d
#2 -> #3: 20d
#3 -> #4: 7d
#4 -> #5: 10d
#5 -> #6: 3d
#6 -> #7: 3d    # sequence repeats from the start
```

The active step is derived from the task's `link` number. Nautical does not store a separate sequence index, so the sequence cannot drift out of sync with the chain.

Panels show the active sequence step and period:

```
Step 3/5 (7d)
```

Upcoming cp sequence rows also show the period used for each row.

6.3 Random periods

`cp` can use a bounded random period:

```
task add "Check trap" cp:"rand(3d..7d)" due:today
```

Each link gets one deterministic pick inside the range, scoped by `chainID`. Separate chains using the same expression get different sequences, while the same chain link resolves to the same period again if Nautical has to recompute it.

Random periods can also be mixed into a sequence:

```
task add "Follow-up cycle" cp:"3d,rand(10d..20d),7d"
```

For “roughly every N days”, use jitter shorthand:

```
task add "Routine inspection" cp:"14d~2d"
```

This means Nautical will pick a deterministic period between `12d` and `16d` for each link. It is equivalent in behavior to a bounded random range, but easier to read for “base period plus/minus spread” schedules.

Nautical shows the selected interval in panels and timelines:

```
Step 2/3 (14d)
```

Use `rand(<duration>..<duration>)` with two dots between the lower and upper bound. For example, `rand(3d..7d)` is valid; `rand(3d-7d)` is not.

This is useful for staged real-world cycles:

- farming or insect lifecycle schedules,
- follow-ups that start close together and then spread out,
- equipment checks after installation or repair,
- training, recovery, or inspection stages.

If you use cp sequences or random cp ranges, `cp` must be configured as a string UDA:

```
uda.cp.type=string
```

If it remains `duration`, Taskwarrior will reject comma-separated or random values before Nautical sees them.

6.4 Wall-clock vs exact-add

- **Multiples of 24h** (e.g., `2d`, `1w`) → keep the same local time as the seed task’s `due:`.

- Add or subtract one second from a multiple of 24 hours to force exact addition, for example `cp:3d-1s`.
- **Other spans** (e.g., `28h`, `33h`) → next due = **end** + **cp** (exact add), so the cadence follows elapsed time from completion.

6.5 Seeding the time

Give the first link a due time; Nautical carries it when appropriate.

```
# Trim the grass every 12 days at 09:00
task add "Trim the grass" due:tomorrow+9h cp:12d

# The next link is going to have a due time of 09:00 regardless if you completed the
current one early or late.
```

6.6 Stopping conditions

- **By count:** `chainMax:5` → stop after 5 links.
- **By date:** `chainUntil:2030-12-31` → stop on/at the last link before that date.
- By manually modifying the chain UDA to **off**. (task 112 modify chain:off)
- By intentional deletion before native `until`. Automatic expiration at `until` advances the chain.

Panels show **links left** and the **final date**.

```
# 33h cadence, cap by count
task add "Calibration checks" cp:33h chainMax:5 due:today+12h

# Daily at noon, cap by date
task add "Deep work block" cp:1d chainUntil:2030-12-20T12:00 due:today+12h
```

7) Anchors (Real-World Patterns)

7.1 Mental model

Anchors point to **calendar positions** (days, weekdays, nth weekdays) and Nautical walks those exact dates. You can combine rules with AND/OR and adjust to business-day reality.

7.2 Weekly anchors

- **Single weekdays:** `mon`, `tue`, `wed`, `thu`, `fri`, `sat`, `sun`.
- **Lists** (comma-separated): `w:mon,fri`.
- **Ranges:** `w:mon..wed` (equivalent to `w:mon,tue,wed`).
- **Shortcuts:** `w:wk` (Mon–Fri), `w:we` (Sat–Sun).

- **Shortcut:** `w:wd` (Mon–Fri).
- **Per-term time:** attach `@t=HH:MM` to **each weekday** if you want **different times per day**, e.g., `w:mon@t=09:00,fri@t=15:00`.
- **Shared group time:** place `@t=HH:MM` after parentheses when every branch should use the same time, e.g., `(w:mon | w:fri)@t=09:00`.
- **Counted random:** `w:2rand` selects two distinct random dates per eligible ISO week.
- **Combine** with logic: AND `+` (e.g., `w:mon + m:1,15`), OR `|`.
- **Time tip:** If you omit `@t=`, Nautical uses the **seed task's due time**.

```
# Mon & Fri (skip missed)
task add "Gym sessions" anchor:w:mon,fri

# Mon/Wed/Fri, but skip every Wednesday in April, July and October
task add "Strength training" anchor:w:mon,wed,fri omit:"w:wed + y:apr,jul,oct"

# Monthly date night, but skip dates listed in holidays.csv inside omit_file_dir
task add "Date night" anchor:'m:rand + w:sat' omit_file:holidays.csv

# Range - Mon..Wed
task add "Study block" anchor:w:mon..wed

# Different times per weekday - 09:00 on Mon, 15:00 on Fri
task add "Split training" anchor:w:mon@t=09:00,fri@t=15:00

# Two random weekdays every week at 09:00
task add "Field inspections" anchor:'w:2rand@bd@t=09:00'

# Only Mondays that are also 1st or 15th (AND)
task add "Friends meeting" anchor:'w:mon + m:1,15' due:today

# Either Friday or Sunday (OR)
task add "Weekend celebration" anchor:'w:fri | w:sun' due:today
```

Tip: You don't have to mention `anchor_mode` in the task addition if you are happy with the default set by the UDA.

7.3 Monthly anchors - by date

- **Specific days:** `m:1`, `m:15`, `m:31`; **last day:** `m:-1`.
- **Lists:** `m:1,15,-1`.
- **Ranges/buckets:** `m:1..7` (days 1–7), `m:22..28`.
- **Shortcuts:** `m:ld` (last day), `m:lbd` (last business day).
- **Business-day ordinals:** `m:5bd` (5th business day), `m:15bd`.
- **Roll modifiers:** `@nbd` (next business day if closed), `@pbd` (previous business day if closed), `@nw` (nearest business day if closed), `@bd` (business days only), and specific weekday rolls `@next-mon`, `@prev-fri`.
- **Business-day offsets:** `@+Nbd` and `@-Nbd` move a matched date by N open dates in the selected calendar; without `bc`, that means Monday through Friday.
- **Per-term time:** attach `@t=HH:MM` to **individual dates** for **different times per date**, e.g., `m:1@t=09:00,15@t=15:00`.

- **Counted random:** `m:3rand` selects three distinct random dates per eligible month.
- **Logic:** AND `+`, OR `|`, and parentheses `( ... )`.
- **Time tip:** Without `@t=`, the seed task's `due:` time is used.

```
# 1st and last day (backfill if missed)
task add "Billing sweep" anchor:m:1,-1 anchor_mode:all due:today

# First business day at 09:00
task add "Payroll" anchor:m:1@nbd@t=09:00 anchor_mode:all due:today

# Two business days before rolled month-end
task add "Month-end preparation" anchor:m:-1@pbd@-2bd due:today

# Mid-month on nearest weekday
task add "Mid-month billing" anchor:m:15@nw

# 5th business day
task add "Supplier payments" anchor:m:5bd anchor_mode:all due:today

# Different times per date - 1st at 09:00, 15th at 15:00
task add "Billing windows" anchor:m:1@t=09:00,15@t=15:00 anchor_mode:all due:today

# Buckets (days 1-7) - pair with logic or random
task add "Focus window" anchor:m:1..7

# Three random weekdays each month
task add "Monthly sampling" anchor:'m:3rand + w:mon..fri'
```

7.4 Monthly anchors - by weekday position

- **Nth weekday:** `m:1mon` ... `m:5sun` (1st–5th).
- **Last weekday:** `m:last-fri`, `m:last-mon`, etc.
- **Lists:** `m:2mon,4thu`.
- **Time:** seed via `due:` or add `@t=HH:MM`.
- **Logic:** AND/OR with other monthly or weekly rules.

```
# 2nd Saturday
task add "Sourdough bake day" anchor:m:2sat

# Last Friday of the month
task add "Design session" anchor:m:last-fri anchor_mode:all due:today

# 1st Wednesday and 3rd Friday
task add "Parent-teacher meeting" anchor:m:1wed,3fri
```

7.5 Yearly anchors

- **Specific dates:** `y:05-20` (May 20 in the fixed MM-DD style).

- **Calendar-day ordinals:** `y:d100` selects the 100th day; `y:d-1` selects the last day. A missing ordinal, such as `d366` in a common year, contributes no date.
- **ISO-week ordinals:** `y:w20` selects all seven days in ISO week 20. Add a weekday when one occurrence is intended, such as `y:w20 + w:mon`. ISO weeks follow ISO week-years and may cross a calendar-year boundary.
- **Reverse ISO weeks:** `y:w-1 + w:fri` selects the Friday of the actual final ISO week, whether that ISO year has 52 or 53 weeks.
- **Ordinal lists and ranges:** use `y:d1,d100,d-1`, `y:d100..d110`, or `y:w10..w13`. Repeat `d` or `w` on both range endpoints. Zero and zero-padded values are invalid.
- **Stepped ISO-year cadence:** a pure ISO-week selector such as `y/2:w1` advances in ISO-year buckets.
- **Lists:** `y:01-15,04-15,07-15,10-15`.
- **Ranges (inclusive):** `y:01-20..01-27` (Jan 20–27), `y:04-20..05-15` (Apr 20–May 15).
- **Random month pick:** `y:10-rand` (one day in October, deterministic per chain).
- **Counted random:** `y:2rand` selects two distinct random dates per eligible year.
- **Quarter aliases:** `y:q1..q4` (quarter window), `y:q1s` (start month), `y:q1m` (mid month), `y:q1e` (end month).
- **Quarter ranges:** `y:q1s..q2s` (start months of Q1–Q2), `y:q1m..q3m` (mid months), `y:q2e..q4e` (end months).
- **Per-term time:** attach `@t=HH:MM` to **individual year-dates** to vary **by month**, e.g., `y:06-01@t=09:00,12-01@t=15:00`.
- **Logic:** AND/OR with other yearly terms.
- **Note:** Business-day rolls mostly apply to monthly day rules; yearly dates pair well with `@t=` for month-specific times.

```
# Anniversary reminder
task add "Anniversary dinner" anchor:y:05-20 anchor_mode:all due:today

# Quarterly review (simple yearly list)
task add "Quarterly review" anchor:y:01-15,04-15,07-15,10-15 anchor_mode:all due:today

# Different times by month - June 1 at 09:00, Dec 1 at 15:00
task add "Seasonal review" anchor:y:06-01@t=09:00,12-01@t=15:00

# A random day in October each year
task add "Spooky surprise" anchor:y:10-rand

# Two random checks each year, selected only from Apr, Jul or Oct
task add "Seasonal audits" anchor:'y:2rand + y:apr,jul,oct'

# Leap day (appears only in leap years)
task add "Leap-day check" anchor:y:02-29

# Last calendar day of each year
task add "Year-end archive" anchor:'y:d-1@t=17:00'

# Monday of ISO week 20
task add "ISO week checkpoint" anchor:'y:w20 + w:mon'

# Quarter-based anchors (advanced)

task add "Quarter start kickoff" anchor:'y:q1s..q4s + m:1'
task add "Quarter end closeout" anchor:'y:q1e..q4e + m:lbd' anchor_mode:all due:today
task add "Mid-quarter check-in" anchor:'y:q1m..q4m + m:15'
```

8) Caps with anchors

Caps stop a chain **by count** (`chainMax`) or **by date** (`chainUntil`). They work with both **Chains** and **Anchors**.

How Nautical counts (Anchors)

- `chainMax` → stops after the **N-th link** (the panel marks *second-to-last* and *last*).
- `chainUntil:YYYY-MM-DD[Thh:mm]` → includes anchors **up to and including** that moment in your **local time**. The final link shown is the last anchor $\leq$ **until**.
- If you set **both**, Nautical stops at the **earliest** of the two limits.

Good to know

- If a link is **overdue**, nothing new spawns until you complete it, intentionally delete it before native `until`, or let native `until` expire it. Completion and expiration advance the chain; intentional deletion stops it.
- **Modes** (`skip`/`all`/`flex`) still apply: caps don't override catch-up semantics.
- Panels show **Links left**, **Last occurrence**, and a timeline that marks (*last link*).

```
# Weekly anchors (Mon/Fri), stop after 6 links

# → exact final date shown in the completion panel
task add "Bootcamp" anchor:w:mon,fri anchor_mode:skip chainMax:6 due:today

# Business-day monthly, stop by date (end of Q4)
task add "AP run" anchor:m:1@nbd due:today chainUntil:2030-12-31T17:00

# Chain (33h cadence), stop after 5 links
task add "Stability checks" cp:33h chainMax:5 due:today+12h
```

9) Recipes library

Chains

```
# Grass trim - every 12 days at 09:00
task add "Trim the grass" due:tomorrow+9h cp:12d

# Vitamins - every 28 hours (exact add)
task add "Take the vitamin" due:today+15h cp:28h

# Insect treatment cycle - staged intervals, then repeat
task add "Insect treatment" cp:"3d,20d,7d,10d,3d" due:today

# Check a trap every 3 to 7 days, deterministic per chain link
task add "Check trap" cp:"rand(3d..7d)" due:today

# Routine inspection roughly every two weeks, plus or minus two days
```

```
task add "Routine inspection" cp:"14d~2d" due:today

# Follow-up cycle with a flexible middle interval
task add "Follow-up cycle" cp:"3d,rand(10d..20d),7d" due:today

# Follow-up schedule - close first, then wider gaps and ends after a full cycle
task add "Client follow-up" cp:"1d,3d,7d,14d,30d" chainMax:5 due:today

# Two-day sprint - stop after 6 links
task add "Focus sprint" cp:2d chainMax:6 due:today+09:00

# Daily deep work until a deadline (keeps 12:00)
task add "Deep work block" cp:1d chainUntil:2030-12-20T12:00 due:today+12h
```

Anchors

Paste this once and replace `<pattern>` with any of the patterns below:

```
task add "Anchor demo"  project:nautical.test anchor:'<pattern>'
```

Tip: attach `@t=HH:MM` **on individual terms** when you need different times, or after parentheses when every branch should share one time: `(w:mon | w:fri)@t=09:00`.

Weekly anchors

```
w:mon → Mondays
w:mon,fri → Mondays and Fridays
w:mon..wed → Monday through Wednesday
w:wk → Mon-Fri (weekday shortcut)
w:wd → Mon-Fri (weekday shortcut)
w:we → Sat-Sun (weekend shortcut)
"w:wd@t=09:00 | w:we@t=11:00" → Mon-Fri @ 09:00 and Sat-Sun @ 11:00
w:fri|w:sun → Fridays or Sundays (OR)
w:mon + m:1,15 → Mondays that are also the 1st or 15th (AND)
w:mon@t=09:00,fri@t=15:00 → Mon at 09:00, Fri at 15:00 (per-term times)
(w:mon | w:fri)@t=09:00 → Monday or Friday, both at 09:00
w/2:mon,tue → every 2 weeks on Monday & Tuesday
w/3:fri → every 3 weeks on Friday
w:mon..fri@t=09:00,17:30 → Mon-Fri with two times per day
w:2rand → two distinct random days each ISO week
```

Monthly anchors - by date

m:1 → 1st day of month
m:-1 → last day of month
m:ld → last day of month (shortcut)
m:1,15,-1 → 1st, 15th, and last day
m:1..7 → bucket: days 1-7 of month
m:5bd → 5th business day of month
m:lbd → last business day of month (shortcut)
m:1@nbd → keep an open 1st; otherwise roll to the next business day
m:1@pbd → keep an open 1st; otherwise roll to the previous business day
m:15@nw → keep an open 15th; otherwise roll to the nearest business day
m:1@t=09:00,15@t=15:00 → 1st at 09:00, 15th at 15:00 (per-term times)
m/2:-1 → every 2 months on the last day
m/3:1 → every 3 months on the 1st
m:1@prev-mon → 1st rolled back to previous Monday
m:1@next-sat → 1st rolled forward to next Saturday
m:1 + y:01..06,12 → 1st of month except Jul-Nov (via inclusion)
m:3rand → three distinct random days each month
m:3rand + w:mon..fri → three distinct random weekdays each month

Monthly anchors - by weekday position

m:2sat → 2nd Saturday of each month
m:last-fri → last Friday of each month
m:1wed,3fri → 1st Wednesday and 3rd Friday
m/2:2sat → every 2 months, 2nd Saturday
m/4:last-fri → every 4 months, last Friday

Yearly anchors

y:05-20 → May 20th (fixed MM-DD style)
y:01-15,04-15,07-15,10-15 → Quarterly markers
y:04-20..05-15 → Apr 20 – May 15 (inclusive range)
y:10-rand → one random day in October (deterministic per chain)
y:2rand → two distinct random days each year
y:2rand + y:apr,jul,oct → two random dates each year selected from Apr, Jul or Oct
y:02-29 → Feb 29 (appears only in leap years)
y:d100 → 100th calendar day of each year
y:d-1 → last calendar day of each year
y:w20 + w:mon → Monday of ISO week 20
y:w-1 + w:fri → Friday of the final ISO week
y:06-01@t=09:00,12-01@t=15:00 → June 1 at 09:00; Dec 1 at 15:00 (per-term times)
y:q1 → Q1 window (Jan-Mar)
y:q1s → Q1 start month (Jan)

y:q1m → Q1 mid month (Feb)
y:q1e → Q1 end month (Mar)

Random gallery

w:rand → one random day each ISO week
w:2rand → two distinct random days each ISO week
(w:rand | w:wed) → Wednesday plus one random day; one date if the draw is Wednesday, otherwise two
m:rand + w:sat,sun → one random Saturday or Sunday each month (one date total)
m:rand + (w:sat | w:sun) → one random Saturday and one random Sunday each month (two dates total)
"w:sun + y:rand" → one random Sunday every year.
(m:rand + y:04-20..05-15) → one random day each month and within Apr 20–May 15 each year
m:rand → one random day each month.
m:3rand → three distinct random days each month
m:3rand + w:mon..fri → three distinct random weekdays each month
m:rand@bd → one random **business day** each month
(m:1..7 + m:rand@bd) → one random business day chosen from the 1–7 bucket
(m:8..14 + m:rand@bd) → one random business day from 8–14 (use with OR buckets)
y:10-rand@t=12:00 → one random October date at 12:00
y:rand → one random day in the Year
y:2rand → two distinct random days each year
"(y:rand + y:01-01..06-30) | (y:rand + y:07-01..12-31)" → One random day in Jan..Jun and another in Jul..Dec.
"y:rand + y:apr,jul,oct" → One random day a year selected from the months Apr, Jul, Oct.
"y:2rand + y:apr,jul,oct" → Two random dates a year selected from Apr, Jul or Oct.

Rolls gallery (quick reference)

@nbd → keep an open date; otherwise roll to the next business day
@pbd → previous business day only when the matched date is closed
@nw → keep an open date; otherwise roll to the nearest business day
@bd → business days only (filter)
@+2bd → two business days later
@-2bd → two business days earlier
@next-mon → roll forward to the next Monday
@prev-mon → roll backward to the previous Monday
@next-sat → roll forward to the next Saturday

Anchor combinations (use + for AND, | for OR, parentheses to group)

'w:mon + m:1,15' → Mondays that are also the 1st or 15th


```
'm:1sat | m:3fri' → either 1st Saturday or 3rd Friday
'(m:1..7 + m:rand@bd) | (m:8..14 + m:rand@bd)' → one random BD in days 1-7 OR 8-14
'w:mon..wed | w:fri' → Mon-Wed or Friday
'w:mon@t=09:00 + m:1..21' → Mondays within days 1-21 at 09:00
'(w:mon | m:last-fri)@t=09:00' → Mondays or the last Friday of each month, all at 09:00
'm:1@nbd + w:fri' → 1st business day that is also a Friday
```

Modes reminder: `anchor_mode:skip` (skip missed), `all` (backfill all), `flex` (skip backlog once, then be strict).

10) FAQ (short)

Q: How do you change the yearly date format?

A: Fixed yearly dates use MM-DD only. Ordinal selectors such as `y:d100` and `y:w20` do not use a date format.

Q: Can you disable caching of the anchors?

A: Use the config file (config-nautical.toml) for the related toggle.

Q: Can you mix `cp` and `anchor` on the same task?

A: Use one engine per task. You can keep separate tasks for each behavior.

Q: Is random really random?

A: It behaves like a random draw but is **deterministic per chain**. Every task gets a fair, stable selection that previews, completion and timelines can reproduce. Counted forms such as `m:3rand` select distinct dates without replacement.

Q: Why didn't the link keep the same time?

A: Only multiples of 24h preserve wall-clock. Odd spans add exactly from **end** (completion time).

Q: What if you forgot to complete a classic recurrence task on the day you actually finished it?

A: Supply the real completion timestamp while completing it, for example `task 42 done end:20260714T180000`. Nautical calculates the next link from that `end` value.

Q: Why not keep the chain as completion time + chain period regardless if it is a multiple of 24h?

A: Preserving wall-clock time for whole-day periods keeps routines stable when completion happens a little early or late. Otherwise, small differences would move the due time on every link.

Q: Can I backfill missed anchors only once, then go strict?

A: Yes - that's `flex` mode.

11) Practical Usage Tips

- **Keep it simple:** prefer concise patterns over massive OR lists.
- **Termux speed:** enable `enable_anchor_cache=true` and use `panel_mode="fast"` when terminal rendering is slow.

- **Quote complex patterns** to avoid shell parsing: `anchor:'(m:1..7 + m:rand) | m:last-fri'`.
-

12) Redundancy and Recovery

This section is focused on production hardening for busy systems (1000+ tasks/hour).

For most personal setups, Nautical works well without enabling every item below.

Operational Knobs

Common knobs:

- `NAUTICAL_DNF_DISK_CACHE=0` disables the on-add JSONL cache (default: enabled).
- `NAUTICAL_EXIT_STRICT=1` makes on-exit return 1 when spawns are dead-lettered or errored (for scripting).
- `NAUTICAL_DIAG=1` prints diagnostics and config search paths.
- `NAUTICAL_DIAG_LOG=1` persists structured diagnostics to `TASKDATA/.nautical_diag.jsonl`.
- `NAUTICAL_DIAG_LOG_MAX_BYTES=262144` caps the diagnostic log before rotation.
- `NAUTICAL_PROFILE=1` emits lightweight timing (stderr).
- `panel_mode="live"` reveals Rich panel rows over a bounded 160 ms interval on interactive terminals.
- `live_panel_duration_ms=0` retains live styling without animation.
- `nautical doctor` reports Rich availability, configured/effective live duration, clamping, and static fallback behavior.
- `panel_mode="fast"` forces plain panel rendering (skip Rich).
- `panel_mode="line"` shows a single summary line inside a compact panel.
- `fast_color=false` disables ANSI in fast panels.
- `max_chain_walk` caps how far chain summaries/analytics run.

Data directory resolution:

- Hooks resolve Taskwarrior data dir from `TASKDATA` or hook argv (`data:` / `data.location:` in Hooks v2).
- `rc.data.location=...` is only injected for hook-spawned `task` calls when that data dir is explicit.

Durable Queue Mode

Diagnostic Log Mode

Set `NAUTICAL_DIAG_LOG=1` to write structured hook diagnostics to `TASKDATA/.nautical_diag.jsonl`.

Use `NAUTICAL_DIAG_LOG_MAX_BYTES` (default `262144`) to cap file size before automatic rotation.

Spawn Queue + Recovery Model

When a chain link is completed, Nautical queues a spawn intent and the on-exit hook imports the child task and then updates the parent `nextLink`. This avoids re-entering Taskwarrior while it holds its datastore lock.

Queue storage is SQLite in

`TASKDATA/.nautical-state/.nautical_queue.db`. Older root-level SQLite, dead-letter, and intent files are migrated into `.nautical-state/` when needed.

If a crash happens between queuing and import, the intent remains in the SQLite queue and is picked up on the next Taskwarrior command. Permanent failures are written to `TASKDATA/.nautical-state/.nautical_dead_letter.jsonl`.

For synced multi-device setups, Nautical treats the next child as a stable chain slot rather than a purely local spawn event:

- The queued child UUID is deterministic for that slot, based on the parent task UUID, `chainID`, recurrence kind, and next link number.
- If two systems complete the same parent before sync converges, both systems derive the same child UUID, so they target the same next task instead of creating two different children.
- During queue drain, on-exit also checks for an already-existing equivalent child by `chainID` + `link` + `prevLink` before importing. If one is found, Nautical links the parent to that task instead of importing another copy.

This prevents future duplicate next links after the updated hooks are installed on all devices that share the database. It does not automatically clean up duplicates that already exist.

On-exit returns 0 by default. Set `NAUTICAL_EXIT_STRICT=1` to return 1 when any spawn was dead-lettered or errored.

Doctor and Recovery Tools

Use the doctor first when a chain looks wrong after sync, hookless clients, manual edits, or interrupted runs:

```
python3 nautical_core/tools/nautical_doctor.py --taskdata ~/.task
python3 nautical_core/tools/nautical_doctor.py --taskdata ~/.task --json
```

`nautical_doctor.py` is read-only. It checks the installation, UDAs, config, queue state, chain metadata, repair opportunities, and completion/expiration reconcile plans. It reports what it sees and suggests the next command; it does not modify tasks.

Recommended workflow:

1. Run `nautical_doctor.py`.
2. If doctor reports `chains.repair_available`, run the chain repair tool in dry-run mode.
3. Review the proposed repairs.
4. Run the same repair tool with `--apply` only if the dry-run is clear.
5. If doctor reports `chains.reconcile_available`, run the reconcile tool in dry-run mode.
6. Review whether it will backfill `nextLink` or spawn a missing next task.
7. Run the reconcile tool with `--apply` only when the dry-run matches what you expect.

Chain repair

Chain repair is for deterministic chain metadata repair:

```
python3 nautical_core/tools/nautical_chain_repair.py
python3 nautical_core/tools/nautical_chain_repair.py --apply
python3 nautical_core/tools/nautical_chain_repair.py --json
```

It can repair safe metadata gaps such as:

- missing or wrong `prevLink`
- missing or wrong `nextLink`
- missing numeric `link` when the value can be inferred without guessing
- singleton root chains that clearly need `link:1`

It will not guess when the chain is ambiguous. Remaining unresolved issues include `why:` lines, for example:

```
issue: review missing_link: 1 task(s) are missing a numeric link
33333333 link - prev:missing1 next:- Check irrigation
why: single-task chain is not rooted at its own chainID
```

Use the JSON output when integrating with scripts. The unresolved task entries include the same reason field.

Completion and expiration reconcile

Reconcile repairs Nautical links that completed or expired without a successor, for example when a web UI, sync service, or temporarily unavailable hook did not run Nautical. It recognizes native expiration only from a deleted task whose `until` is at or before its recorded `end`; an earlier intentional deletion is not advanced.

```
python3 nautical_core/tools/nautical_reconcile.py
python3 nautical_core/tools/nautical_reconcile.py --apply
python3 nautical_core/tools/nautical_reconcile.py --json
```

Reconcile can:

- backfill a missing parent `nextLink` when the next child already exists
- spawn the missing next task when no child exists and the recurrence can be computed
- advance an expired occurrence from its original `due` or scheduled-only `scheduled`
- identify a legitimate final link when `chainMax` or `chainUntil` has been reached
- report errors when the next recurrence cannot be computed safely

Dry-run output explains the plan before doing anything:

```
backfill nextLink: 11111111 chain cid link 1 · hookless completed parent
reason: next link already exists
next link: 2
existing child: 22222222
```

With `--apply`, reconcile may create tasks. Use doctor and dry-run first.

Health Check and Alerting

Use the local health checker to watch queue/dead-letter pressure and lock contention (JSONL + SQLite queue metrics):

```
python3 dev_tools/nautical_health_check.py --taskdata ~/.task --json
```

Exit codes:

- `0` healthy
- `1` warning
- `2` critical

Recommended rollout:

1. Enable `NAUTICAL_DIAG_LOG=1` first.
2. Keep `NAUTICAL_EXIT_STRICT=0` during bake-in.
3. Monitor dead-letter and queue metrics.

4. Flip `NAUTICAL_EXIT_STRICT=1` once stable.

Periodic alert example:

```
python3 dev_tools/nautical_health_check.py --taskdata ~/.task --queue-crit-bytes 524288
python3 dev_tools/nautical_health_check.py --taskdata ~/.task --queue-db-crit-rows 200
```

Automation templates in `dev_tools/ops/`:

- `nautical-health-check.crontab`
- `nautical-health-check.service`
- `nautical-health-check.timer`
- `nautical_health_check_cron.sh`
- `README.md` (install steps)

Performance Checklist

- Enable `enable_anchor_cache=true` for file-backed anchor hints; its built-in default is off and the shipped config template enables it.
- Use `panel_mode="fast"` on slow terminals or mobile.
- If you see slowdowns, run `NAUTICAL_PROFILE=1` for a short session.
- For heavy workloads, monitor queue growth and dead-letter pressure with the `queue-status` and `health-check` commands.

Load Testing

```
python3 dev_tools/load_test_nautical.py --tasks 2000 --concurrency 4
python3 dev_tools/load_test_nautical.py --ramp --ramp-start 200 --ramp-step 500 --ramp-max 10000 --concurrency 16
python3 dev_tools/load_test_nautical.py --ramp --done-only --ramp-start 200 --ramp-step 500 --ramp-max 10000 --concurrency 16
python3 dev_tools/load_test_nautical.py --rate-ramp --rate-secs 30 --rate-start 5 --rate-step 5 --rate-max 100
```

Mode summary:

- Batch: fixed number of adds (and optional dones), report latency stats.
- Ramp: increase task count per stage until thresholds are hit.
- Done-only: measure on-modify performance by completing tasks created in the stage.
- Rate-ramp: increase target ops/sec and report throughput and latency limits.

Self-check and diagnostics

```
python3 nautical_navigator.py --self-check
NAUTICAL_DIAG=1 python3 nautical_navigator.py --self-check
```

Anchor explain / validate

```
python3 nautical_navigator.py --explain "m:last-fri"
python3 nautical_navigator.py --validate "w:mon..fri@t=09:00,17:00"
```


Local occurrence queries

Tools that need Nautical's recurrence results can use the read-only local query contract instead of importing Nautical internals or reimplementing scheduling. The command emits exactly one versioned JSON document on stdout.

```
# All future schedule matches for one task on the requested local date.
nautical query occurrences --request '{
  "selector": {"uuids": ["4f2f1c2e-0000-0000-0000-000000000001"]},
  "from": "2026-08-24",
  "to": "2026-08-31",
  "omission_policy": "exclude"
}'

# Ask for the next five matches instead of supplying an end date.
printf '%s\n' '{
  "selector": {"chainID": "176f5c68"},
  "from": "2026-08-24T09:00:00+03:00",
  "count": 5
}' | nautical query occurrences
```

The response has schema `nautical.query.occurrences`, version `1`, and a `basis` of `schedule`. It reports calendar matches calculated by Nautical's authoritative scheduler; it does not promise that lifecycle hooks have already created those Taskwarrior children. Use `status` and the structured `failure` code to distinguish an empty schedule, an exhausted search, an absent task, and an unavailable dependency. Returned occurrences include both configured local time and the equivalent UTC instant.

The same JSON can be consumed without importing Nautical:

```
# Shell/jq: print local occurrence times.
nautical query occurrences --uuid 4f2f1c2e --from 2026-08-24 --count 5 \
  | jq -r '.results[].occurrences[].local'

# Python: inspect structured statuses and failures.
nautical query occurrences --uuid 4f2f1c2e --from 2026-08-24 --count 5 \
  | python3 -c 'import json,sys; p=json.load(sys.stdin); print(p["status"]); print(p["failure"])'

# Plain shell: request a read-only projected successor.
nautical query next --chain-id 176f5c68 --from 2026-08-24 --count 1

# All active Nautical tasks, bounded to one day.
nautical query occurrences --all --from 2026-08-24 --to 2026-08-24
```

`next` is a separately named read-only projection and includes lifecycle metadata; it does not create a child or modify Taskwarrior. Requests are bounded by `to` or `count` and are subject to the advertised hard caps. A truncated or exhausted result is not a complete schedule. Optional provider or configuration failures remain structured `unavailable` results and include retryability without exposing tracebacks or private paths.

For task selectors, occurrence collection also respects the task's current `due` value, falling back to `scheduled` when no due is present. Calendar matches before that current task reference are not returned; this prevents a future chain link from appearing to be active in an earlier year.

For mode-aware daily progress, evaluate `next` at an explicit instant:

```
nautical query next --uuid TASK_UUID --at 2026-08-24T15:00:00+03:00
```

Anchor results include `lifecycle.daily_instances` with the local date, `total`, `current_position`, `missed`, and `upcoming` counts. Skip-mode results also include the exact `missed_occurrences` and the selected next slot. The explicit timestamp keeps this lifecycle projection reproducible.

Dead-letter handling

The current queue is SQLite-backed. Do not append new work to legacy JSONL queue files; use the SQLite queue and the queue-status tool instead.

```
python3 nautical_core/tools/nautical_queue_status.py --taskdata ~/.task
# Explicitly prune only acknowledged intents older than the retention policy:
python3 nautical_core/tools/nautical_queue_status.py --taskdata ~/.task --prune-acknowledged
```

`nautical reconcile --apply` also performs this cleanup opportunistically at most once per day when expired acknowledgements or an oversized outbox make it worthwhile. Use `--no-housekeeping` to skip that maintenance for one run.

1. Run the doctor and queue-status tools to identify the failed intent.
2. Inspect `TASKDATA/.nautical-state/.nautical_dead_letter.jsonl` for its reason.
3. Correct the underlying configuration, file, permission, or Taskwarrior error.
4. Run reconcile in dry-run mode. For a completed or expired parent with no child, reconcile can safely plan and create the missing link.
5. Apply reconcile only when its plan matches the intended chain.

Break-glass Mode

If hooks misbehave or Taskwarrior input format changes, bypass hooks temporarily:

```
task rc.hooks=off <command>
```

Environment and Files

- `TASKDATA` points to the Taskwarrior data directory; Nautical reads/writes queues and locks there.
- `NAUTICAL_CORE_PATH` overrides the core load path (dev/tests); it can point to either a directory containing `nautical_core/` or directly to `nautical_core/__init__.py`.
- Hooks also read Hooks v2 argv tokens (`data:` / `data.location:`) to resolve Taskwarrior data dir when `TASKDATA` is not set.
- Hook-spawned `task` subprocesses only force `rc.data.location=...` when data dir is explicit (env/argv), avoiding bad fallbacks on split config/data setups.

- Hook state lives under `TASKDATA/.nautical-state/`:
 - `.nautical_queue.db` plus SQLite WAL/SHM sidecars
 - `.nautical_dead_letter.jsonl`
 - `.nautical_spawn_intents.jsonl`
- Hook locks and lock-failure markers live under `TASKDATA/.nautical-locks/`:
 - `.nautical_spawn_queue.lock`
 - `.nautical_dead_letter.lock`
 - `.nautical_parent_nextlink.*.lock`
 - `.nautical_spawn_queue.lock_failed`
 - `.nautical_spawn_queue.lock_failed.count`

13) Troubleshooting

Nautical is a complex system handling intricate calendar logic, time zone calculations, and state transitions across hundreds of edge cases. While it's been tested rigorously across various scenarios, the surface area for bugs remains significant.

If you encounter unexpected behavior:

- Check your pattern syntax against the reference sections above
- Verify your UDA configuration matches the expected setup
- Review the panel output for clues about what Nautical computed

When something breaks:

Open a GitHub issue with:

- Your pattern or chain configuration
- The unexpected behavior (what you expected vs. what happened)
- Relevant panel output or task details
- Your Taskwarrior version and environment (desktop/Termux/etc.)

The more context you provide, the faster we can isolate and fix the issue.

[Repository](#)

14) Support the project

I've been working on this project through the darkest time of my life, where I'm still at when I'm typing this.

The only thing kept me going was the promise I've made to my-self to never give up.

If Nautical has improved your workflow and you'd like to support continued development:

[Buy me a book](#), [PayPal](#) or [GitHub](#)

Your support helps sustain the time invested in building, testing, and maintaining systems like this. Every contribution is greatly appreciated.